

OPERATING SYSTEM OF SYSTEM 250

D.M. England

Plessey Company Ltd.
Taplow Court, Maidenhead,
Berks. England.

Abstract

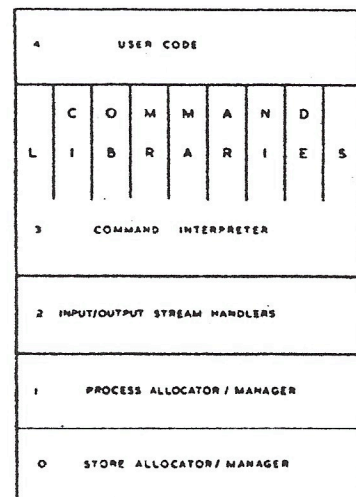
The paper describes the Operating System of System 250 in terms of a series of abstractions achieving logical facilities convenient to the applications programmer and a unique Software interface between applications and operating systems code.

LEVELS AND ABSTRACTIONS

1. In a now classical paper on computer operating systems, Edsger W. Dijkstra postulates an operating system constructed as a series of layers or levels. Each level has two significant characteristics, the present paper being more concerned with the second:-
 - (1) It offers a set of facilities to all higher levels in the operating system's hierarchy, enabling it to be built up layer by layer.
 - (2) It achieves some abstraction; some physical hardware entity (e.g. storage devices) is transformed into some smooth, logical, abstract entity (e.g. virtual memory) that is, moreover, altogether easier to manipulate.
2. The effect of Dijkstra's abstractions is as follows:-
 - (1) At the bottom of the hierarchy rests the responsibility for scheduling processor time, allocating processing power to the competing programs able to run. Above this level, each program believes that it has a processor to itself; so long as it is furnished with the processing power it needs, the fact that it shares a processor with others can be disregarded.
 - (2) The "segment controller" draws a veil over the boundary between the main store and the disk. Above this level, a program simply asks for a "store segment" to be allocated. When the program tries to use this store segment, the segment controller ensures that it is available in main store. When an allocated store segment is not being used the segment controller transfers it onto the disk until it is next used.
 - (3) Programs higher than level 2 can converse with the operator via the console keyboard. Each program believes itself to be in sole communication with the operator and oblivious of simultaneous conversations.
 - (4) The software at level 3 is concerned with buffering input data streams and unbuffering output data streams, between the actual peripheral devices and the programs which process the data. Above this

level actual communication devices lose their identity, and programs communicate with the outside world through the medium of "logical communication units".

- (5) The independent sets of user programs can now reside in their private worlds, furnished in abstractions, whilst in reality they play a game of cox and box with the real resources of the processing system.
3. Threefold advantage is to be gained by this series of abstractions:-
- (1) The system is enabled to perform a variety of jobs simultaneously and without any interaction between them, making maximum use of the physical resources available.
 - (2) The user programs, which are after all the *raison d'être* of any processing system, are rendered independent of system hardware and thus robust to changes in the configuration and capacity of the equipment employed.
 - (3) Simplification of the user programs is achieved to the extent that they are concerned solely with the manipulation of convenient abstract entities whose behaviour is entirely logical and consistent.



1. BROAD STRUCTURE OF SYSTEM 250
OPERATING SYSTEM

AIM OF THE PAPER

4. The PP250 Operating System is constructed as a series of levels, a hierarchy of subroutines, in a manner similar to that suggested by Dijkstra. Fig. 1 depicts in broad terms the structure of the PP250 Operating System, each of its layers being discussed in subsequent paragraphs. This paper is more concerned, however, with the corresponding abstractions; to describe how these are achieved is to explain the principles upon which the system is based. Thus, given the hardware mechanisms described in companion papers, the aim of this paper is to show how the physical resources of the system, processors, storage media, and peripheral devices, are transformed into abstract resources, processes, virtual memory, and data streams.

PROCESSORS TO PROCESSES

5. To show how CPUs suffer loss of identity it is necessary to anticipate the discussion on virtual memory to the extent of assuming a system of dynamic storage allocation. This implies the existence of privileged code, in the dungeons of the operating system, with the power to create on demand blocks of the requisite size and deliver capabilities for them.

6. A program may be thought of as a static structure of code blocks and constant data blocks contained in a network of constant capability blocks. It must have a particular start point, a particular code block within this network. Apply a CPU at this start point and it executes code, it processes input data, it asks for data blocks and is delivered capabilities, and so it has to ask for capability blocks in which to store them. In short, it constructs its own private sub-network of capabilities, a private data structure unique to this run of program.

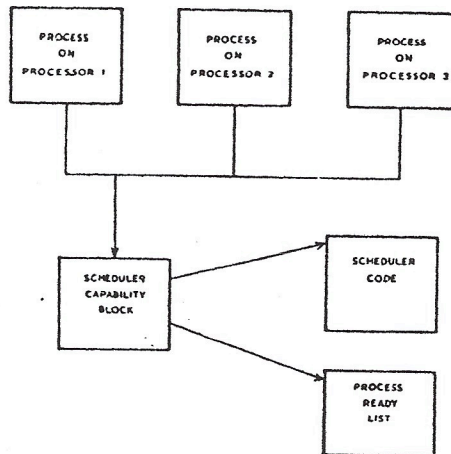
7. Apply another CPU at the same start point, perhaps at the same time, and although it obeys the same code it asks for its own blocks and constructs a separate data structure. Allow discontinuities in each execution of the program, e.g. while waiting for more input data, and the two runs can be performed on the same CPU, whose power is shared or scheduled between the two runs.

8. Two conclusions can now be reached:-

- (1) It is possible to distinguish between execution of a program and the program itself. Although there may be many more or less simultaneous executions of the same program, the program itself is singular. The term used for the execution of a program is a "process" and that for the kind of program which can have many simultaneous executions is a "reentrant program". In a telephone switching system, for example, there may be many processes, each servicing a different telephone call, but all obeying the same reentrant telephone switching algorithm.
- (2) It is immaterial to a set of processes whether each has its own CPU or all are scheduled on the same CPU, so long as each receives a requisite amount of processing power.

9. As a system normally contains a number of different programs, so it is possible to have processes of different types simultaneously in existence.

10. CPUs have now almost disappeared, short of explaining the mechanism for scheduling processes in a multi-processor environment (see Fig. 2). The action of this mechanism is to select another process to run on a CPU when the current process on that CPU comes to an end or has to await some event, e.g. arrival of more input data:-



2. REENTRANT PROCESS SCHEDULER

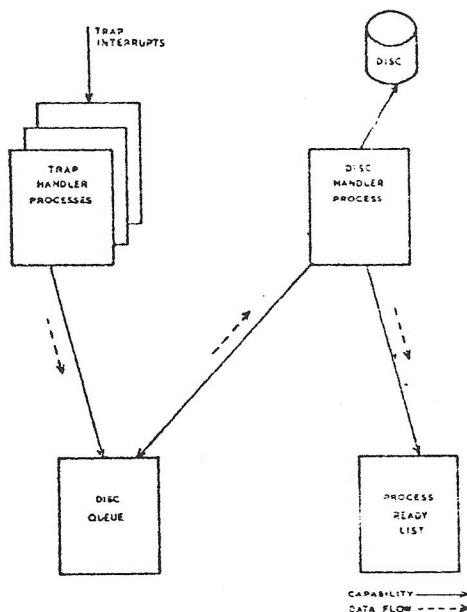
- (1) It is part of the action of every process which runs on a CPU select the next process to run on that CPU by executing a "process scheduler" subroutine.
- (2) The process scheduler selects the first item in a "process ready list" of processes that are in an executable condition.
- (3) As there is only one process ready list for the entire system, it provides a common work pool for all CPUs. A process might run on any CPU, whichever selects it when it reaches the head of the process ready list. Further, where there are discontinuities in the running of a process, different parts of its run might be performed on different CPUs.

11. It may now be seen that CPUs do no more than furnish processing power to processes, the number of CPUs in any configuration being dictated solely by its total processing requirement, the processes themselves being insensitive to the number of CPUs present and the actual CPUs on which they run. Like Dijkstra's processor allocator, the process scheduler furnishes to each process an abstract CPU which meets its total processing requirement. The abstraction of CPUs is thus achieved.

VIRTUAL MEMORY - STORAGE MEDIA TO BLOCKS

12. The purpose of this particular abstraction is to eliminate the distinction between main storage and backing storage and enable a process to be concerned solely with store blocks of the requisite size and type of access. The subject is a large one and can only be given cursory treatment in a paper at this level.

13. The aim of the virtual memory scheme is for blocks to be transferred between main store and disk automatically, without the processes using these blocks being conscious that any such transfers are taking place. The only requirement that a process has is that when it attempts to use a block, the block should automatically be transferred into main store if it is not there already. A simple hardware mechanism is used to detect an attempt to use a block which is not currently in main store. This causes an interrupt into a "trap handler" process (fig. 3) of which there is one per CPU.



3. TRAP AND DISC HANDLERS

14. The action of the trap handler process is to queue the trapped process, with an indication of the required block, for action by the "disk handler" process. The disk handler initiates transfer of the required block into main storage space allocated for the purpose. It then enters the trapped process in the process ready list. When the process is rescheduled it successfully reattempts the trapped instruction, and proceeds to use the block concerned as if nothing had happened.

15. When a process requires a new block of storage, it calls a "store allocator" subroutine, specifying the length and access required. The store allocator allocates appropriate space on the disk. It delivers to the process a capability for the block thus created. An attempt to use the block would, according to the mechanism described above, result in the block being transferred from disk into main storage space allocated for it. In fact, the actual transfer is in this case short-circuited, as the block does not yet contain any useful information.

16. When a process has no further need for a block, it overwrites all its capabilities for the block. The system may reuse disk space occupied by a block when no capabilities for it exist, as the block is then inaccessible to all processes.

17. In this summary of the virtual memory scheme, it remains to outline the algorithm used to select blocks for transferring out:-

- (1) Essentially, the algorithm employed is "longest in", it selects for transferring out the block which has been longest in main store.
- (2) However, the algorithm permits certain critical blocks to be locked down in main store, e.g. those used by the disk handler.
- (3) When a block can be recognised as having remained unchanged while in main store, i.e. if access to it is limited to read data, and/or execute, or read capability, the space it occupies is deallocated without any actual transfer out having to take place.

18. Although the mechanisms required to achieve a virtual memory are quite involved, its use is simplicity itself; a process simply executes the store allocator when it requires more store and overwrites all copies of capabilities for blocks it no longer requires. This discussion makes no distinction between data type and capability type blocks. It makes the tacit assumption, therefore, that capability blocks can be held on disk, indeed that the capability network extends from the address space of main store to the address space of the disk, regardless of the physical boundary between them. It is essential for this to be so for a virtual memory, the abstraction of storage devices, to be achieved.

DEVICES TO DATA STREAMS

19. The purpose here is the abstraction of non-interactive devices, paper tape readers, paper tape punches, lineprinters, etc. in favour of "data streams". A process may then be concerned simply with its input and output data regardless of the actual devices employed.

20. When a process requires to input or output, it must first execute a "stream allocator" subroutine, supplying the stream name, in keyboard characters, as a parameter. Correspondingly, a human may use a keyboard command to ascribe that name to a device. The action of the stream allocator is to allocate to the process the device thus named. The process may then either input data from the stream or output data to it by executing a "get data" or "put data" subroutine respectively, some suitable unit of data e.g. 100 words, being thus transferred.

21. The process is insensitive to the device, even the type of device, used to transfer its data. The "put data" or "get data" subroutine simply passes a unit of data to or from a device handler process associated with the device concerned and responsible for the physical transfer of data.

22. The human may ascribe a name to a data file within the virtual memory instead of a physical device. The action of the stream allocator is then to allocate the data file to the process as its data stream. Physical transfer between the data file and a device is then wholly disassociated from the process, to achieve a conventional "offlining" facility. The process, however, still employs "get data" and "put data" as discussed above to communicate with its streams. Thus, a process is insensitive not only to the devices providing the source and destination of its data but also to whether or not that data is off-lined. Data streams correspond to Dijkstra's "logical communication units".

FORM OF A LOGICAL RESOURCE

23. To be complete, this paper must answer the question "What, then, is a logical resource?" In the case of a store block the answer is self evident: the store block is itself the resource. A more complex resource consists of a collection of store blocks which can contain, in some meaningful format, data describing the resource. To take an example, the format for a process must provide for its start point, priority, space to preserve register values on interrupt or suspension, etc. A logical resource, then, is nothing more than a data structure, of which a single block is the simplest example.

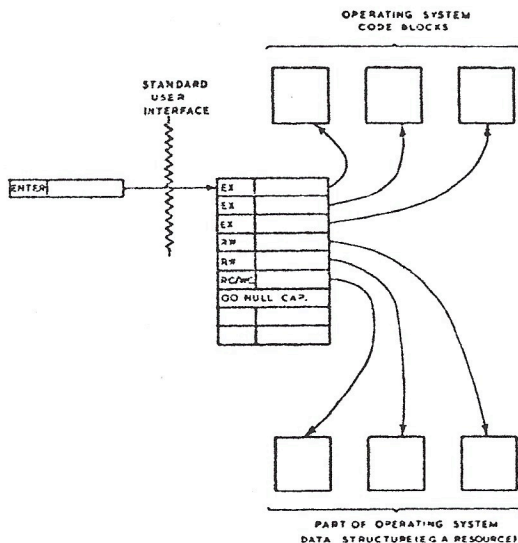
24. The paper now goes on to show that, just like a store block, any logical resource is represented to the user by a single capability. The paper describes how the operating system is called and how abstract resources are represented, allocated, and operated upon. It shows the realisation of a standard software interface.

CALLING THE OPERATING SYSTEM

25. Calls to the operating system are invariably achieved by means of an "enter type" capability, as described in companion papers. This means that, in order to call the operating system, each process has to be furnished with an enter type capability for operating system subroutines (fig. 4). This enables the process to execute code blocks within the operating system, which thus carries out the required operations. Two points should be observed:-

- (1) The enter mechanism automatically denies to the calling (user) subroutine access to the called (operating system) subroutine's data structure, thus protecting the operating system from corruption by user processes.
- (2) Thus, there is no formality involved in a call to an operating system subroutine, which is called just like any other. Unlike conventional systems, no special mechanism is required to enter the operating system, which in turn requires no "privileged mode" of operation.

26. Each process is furnished with an enter type capability for a "resource allocator" within the operating system. This gives the process access to a number of code blocks, each one being concerned with the allocation of a particular resource-type, the "store allocator", the "stream allocator", etc. execution of which causes a logical resource of the corresponding type to be created and allocated to the process, as described below.



4. USER INTERFACE/ACCESS TO A RESOURCE

RESOURCE REPRESENTATION AND ALLOCATION

27. Execution of the store allocator causes a store block of the required size to be created and a capability with the required access to be delivered. This type of resource might be thought of as a special case; once allocated and represented by a capability, a store block can be operated upon directly by means of CPU instructions, whereas all other resource-types can only be operated upon by calling the operating system. The abstract machine, however, is insensitive to such distinctions; it is no more than a convenience of implementation that operations which are performed on simple data structures, individual

blocks, are achieved by hardware, whereas those on more complex data structures are achieved by software. An abstract resource is no more than a particular data structure of blocks, the action of its resource-type allocator being as follows:-

- (1) It creates an appropriate resource (data structure) and delivers a capability for it. This is a direct parallel with the store allocator.
- (2) In this case, however, the capability delivered is an enter type capability enabling suitable code blocks to be executed to perform operations on the resource (data structure).
- (3) Here is the curious and intriguing point, that this is an interface to the operating system specially created by the resource-type allocator to perform operations on the particular resource concerned. The resource-type allocator obtains a capability block and inserts into it execute type capabilities for the standard code blocks used to perform these operations and for the constituent blocks of the resource's data structure. Fig. 4 still applies.

A STANDARD, DYNAMIC, AND ADAPTIVE INTERFACE

28. It should be observed that, when a user process employs its enter type capability to execute a resource-type allocator, it is (excepting storage) delivered another enter type capability with which to perform operations on the resource thus allocated. The two invariable rules constituting a standard user interface with the abstract machine can now be stated formally:-

- (1) A logical resource is always represented by a capability, which can be used to perform permissible operations on that resource.
- (2) A call of the operating system is always achieved by means of an enter type capability.

29. Although standard, the user interface can now be seen as completely dynamic. When a user process requests a resource, it is furnished with its own private interface to the operating system, which it can use to perform operations on that resource. Further, this dynamic interface adapts to a user's requirements; as his processes request resources of various types, the delivery of corresponding capabilities extends the scope of his user interface to provide just those operating system facilities he requires.

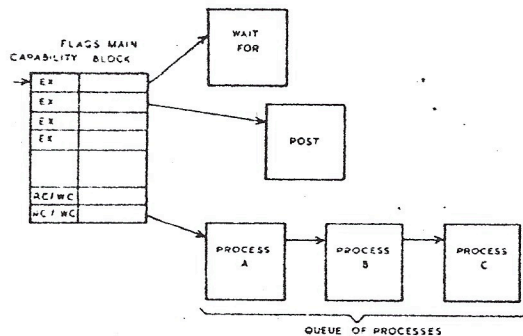
SYNCHRONISING FLAGS

30. A further resource (data structure) is normally employed for synchronising processes and communicating between them. This resource is called a "flag", and may be created by executing a "flag allocator". The problem which the flag mechanism is designed to overcome is that, while processes perform asynchronously, they nevertheless require to synchronise the transfer of information between them. The concept of flags is best understood in terms of the operations that can be performed on them:-

- (1) When a process has a message to send to another process, it "posts" the message "on a flag" for which both have a capability, it executes a block of code that records the message within the flag's data structure.
- (2) The receiving process is delivered the message on request, it executes a block of code that extracts the message from the flag's data structure. However if the receiving process requests the message prior to its being posted, it is suspended until the sending process has performed the post operation. The receiving process is said to "wait for" a message "on a flag".

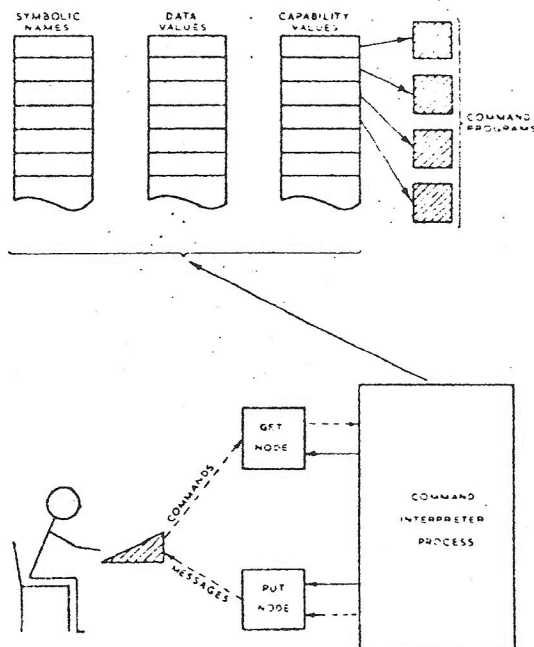
31. Thus, the operations that can be performed on a flag are to post a message and to wait for a message. Although the processes

involved perform asynchronously, the flag mechanism imposes synchronisation on the transfer of messages between them by ensuring that posting and delivery occur in the logically correct sequence. The flag concept permits a number of processes to wait for (fig. 5) or post messages on the same flag. Such messages would be waited for in turn and delivered in posting order. Also, a process may wait for a message on more than one flag, where it can expect one of a number of possible events, e.g. in a telephone switching process, a message indicating the next dialled digit, the return of the handset to its rest, or a timeout.



5. BROAD DATA STRUCTURE OF FLAG THE HUMAN INTERFACE

32. When a human approaches a terminal and operates the "attention key", this causes a "command interpreter" process to be created, the purpose of which is to interpret and obey each command subsequently typed at the terminal keyboard (fig. 6). As the only action the system is able to take in obeying



6. SYMBOL TABLE

such commands is to allocate and manipulate the resources of the abstract machine, a command must be compounded from a basic set of commands, each of which correspond to an abstract machine facility. This basic command set extends the abstract machine to the terminal, where its facilities are then available for the human to create and manipulate abstract resources at will.

33. When created, a resource is represented within the system by a capability. When the human creates a resource at a terminal he gives it a symbolic name. These are respectively the internal and external representation of the resource; each reference that the human makes to the resource is replaced within the system by its associated capability. Such associations are recorded within a "symbol table", which is part of the data structure of the command interpreter process.

34. Each command name is also recorded in the symbol table, its associated capability defining a "command program" which is executed by the command interpreter in order to obey the command. Whenever the human types the command, to create or manipulate an abstract resource, it is this command program which calls the associated abstract machine facility on his behalf.

35. The human is permitted to generate new commands, with their own symbolic names, being compounded of existing commands. This enables him to manipulate a number of resources with a single command, which may then be expressed solely in terms of the work he requires the system to undertake and independently of the abstract resources actually involved.

THE USER

36. To conclude this paper, the main concerns of a system user, in expediting the design of his application, may be summarised as follows:-

- (1) The standard software interface is as pertinent to applications programs as it is to the operating system. It is a task of the user, therefore, to identify logical resources (data structures) convenient to his application, e.g. a telephone subscriber, and the operations to be performed on those resources, e.g. send ring tone. He may then proceed to write corresponding resource-type allocators and the code blocks which achieve the required operations.
- (2) The user must determine the types of processes required to achieve his application and the criteria by which such processes are created, e.g. a telephone switching process might be created when it is detected that a subscriber has raised his hand-set.
- (3) There may be a range of applications personnel who are required to communicate with the system for different purposes, e.g. operational staff, maintenance staff. The user must determine the terminal facilities required by such personnel and establish corresponding command structures. He may then proceed to define the command programs which achieve these facilities, using for this the command generation commands.

Acknowledgement

I would like to thank the many colleagues on whose work this paper is based and the Directors of the Plessey Company for permission to publish it.